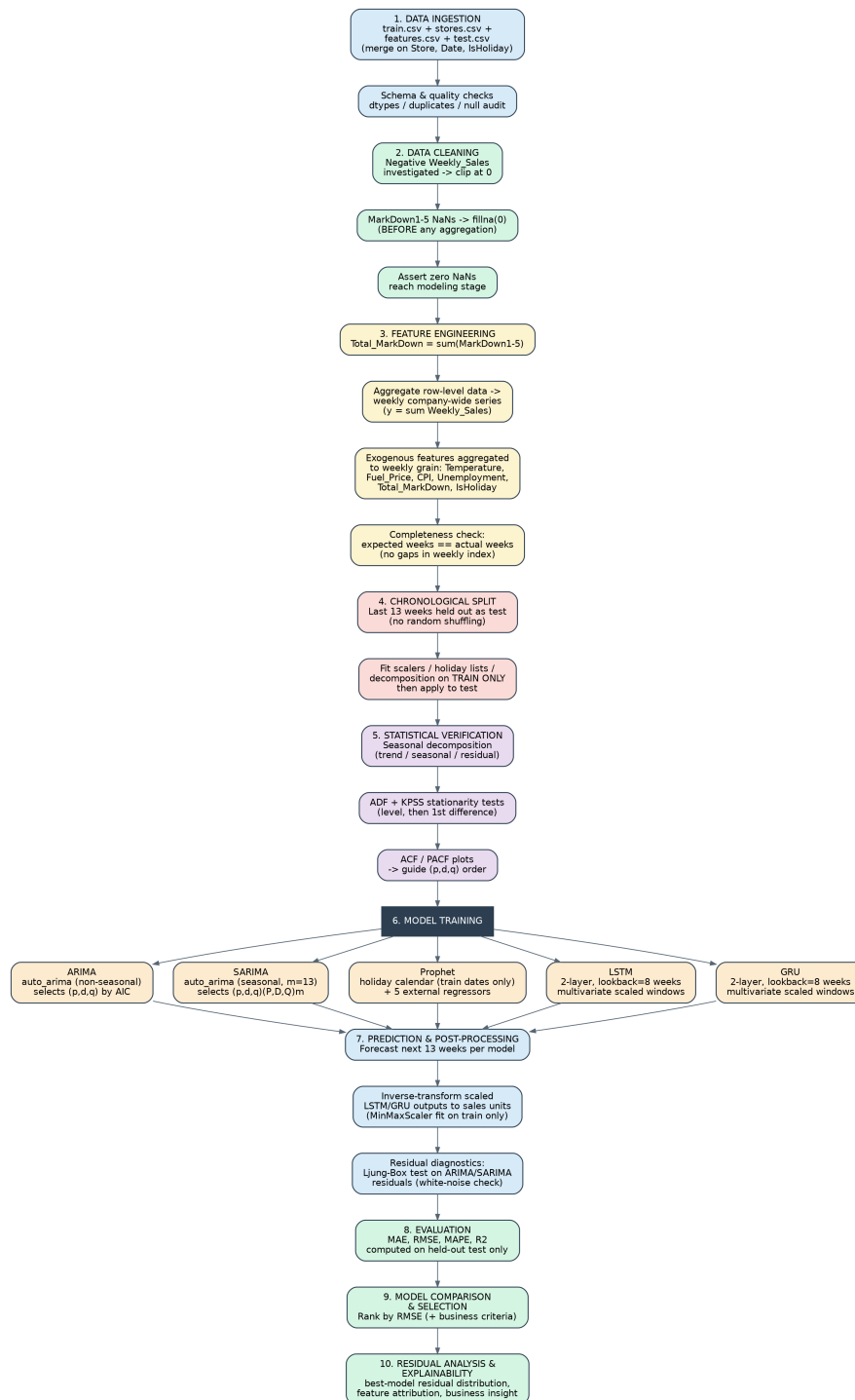


Retail Demand Forecasting

Walmart Store Sales — Time-Series Modeling Pipeline

Process Flow Blueprint | Best Model Selection & Reasoning | Model Evaluation & Justification | Model Explainability



Models covered: ARIMA · SARIMA · Prophet · LSTM · GRU | Forecast target: company-wide weekly demand | Holdout: last 13 weeks (chronological)

Table of Contents

- 1. Process Flow Document
 - 1.1 Data Ingestion
 - 1.2 Data Cleaning & Transformation
 - 1.3 Feature Engineering Pipeline
 - 1.4 Model Training & Validation
 - 1.5 Prediction & Post-Processing
- 2. Best Model Selection & Reasoning
 - 2.1 Comparative Performance Table
 - 2.2 Trade-off Analysis
 - 2.3 Business Context
 - 2.4 Limitations & Improvements
- 3. Model Evaluation & Justification
 - 3.1 Evaluation Metrics (incl. WMAE)
 - 3.2 Validation Strategy — Why Not Random Splits
 - 3.3 Residual Analysis
 - 3.4 Robustness Testing
- 4. Model Explainability
 - 4.1 Global Interpretability
 - 4.2 Local Interpretability
 - 4.3 Business Interpretation
 - 4.4 Model Fairness

1 Process Flow Document

This section is the technical blueprint of the pipeline: a data scientist should be able to reproduce the notebook end-to-end from this description alone, without reading the source code. The full flow — from raw CSVs to a selected, explainable forecasting model — is shown below, followed by a stage-by-stage written specification.

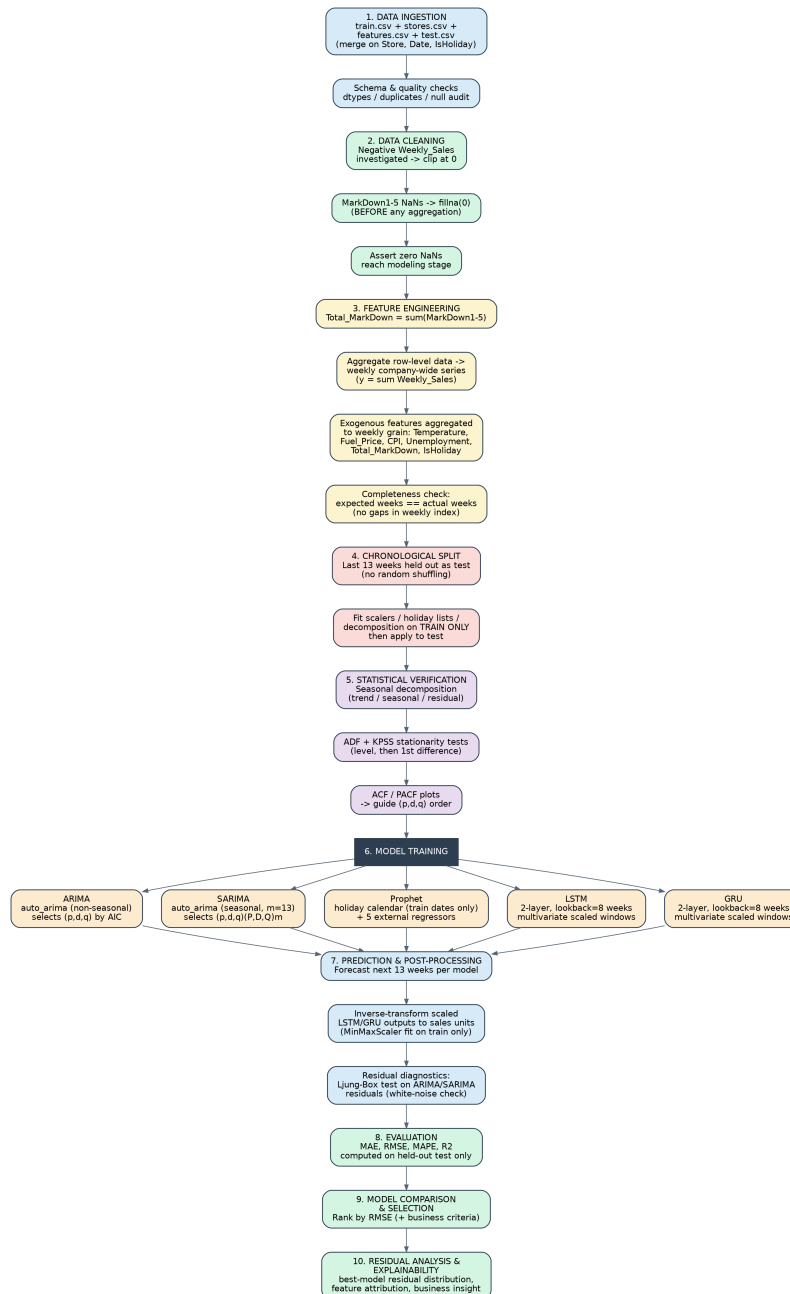


Figure 1. End-to-end process flow: ingestion → cleaning → feature engineering → chronological split → statistical verification → 5 parallel model branches → prediction/post-processing → evaluation, selection and explainability.

1.1 Data Ingestion

Four source files are loaded independently and merged into a single row-level frame:

- **train.csv** — historical Store × Dept × Week sales (target: Weekly_Sales).
- **stores.csv** — static store attributes: Type (A/B/C) and Size.
- **features.csv** — time-varying macro/promotional signals: Temperature, Fuel_Price, Markdown1–5, CPI, Unemployment, IsHoliday.
- **test.csv** — the competition's future Store × Dept × Week grid (not used for training in this notebook; the internal chronological holdout is used instead — see §1.4).

Merge strategy: `train.merge(features, on=[Store, Date, IsHoliday], how='left')` followed by a left join to `stores` on `Store`. Left joins are used deliberately so that no sales rows are dropped if a store/date combination is momentarily missing from the features table. The merged frame has 421,570 rows × 16 columns.

Validation performed immediately after merge: dtype audit (`df.info()`), a full null-count pass per column, and an explicit duplicate-row check (`df.duplicated().sum()` → 0 duplicates found). Only after these checks pass does the frame proceed to cleaning.

1.2 Data Cleaning & Transformation

Negative Weekly_Sales.

1,285 rows (0.305% of data) carry negative Weekly_Sales, ranging from −4,988.94 to −0.02, representing weeks where recorded merchandise returns exceeded new sales for that store/department. These are *investigated before being acted on* rather than silently clipped or dropped. The chosen treatment is **clip at 0** rather than row deletion, because deleting rows would create gaps in the weekly time index — every downstream model in this notebook (ARIMA/SARIMA/Prophet/LSTM/GRU) assumes a complete, regularly-spaced weekly series, so preserving the row while flooring the value at zero keeps the calendar intact.

Missing values (Markdown1–5).

Between 26% and 74% of Markdown1–5 values are missing (promotions that simply did not run that week). These are filled with 0 (`fillna(0)`) — a domain-informed choice, since a missing markdown record means "no promotion," not "unknown promotion." Critically, this fill happens **before** any aggregation or feature construction, and is immediately followed by an assertion (`assert df[markdown_cols].isnull().sum().sum() == 0`) so that no NaN can silently propagate into later steps.

Duplicate rows.

Checked explicitly post-merge; none found, so no de-duplication logic was required.

No categorical encoding is required at the aggregate-series stage: Store Type is used only descriptively in EDA (Figure: "Average Weekly Sales by Store Type"), and does not enter any of the five forecasting models, since all five forecast a single company-wide series.

1.3 Feature Engineering Pipeline

Feature	Construction	Rationale
Total_MarkDown	Row-wise sum of Markdown1-5 (post <code>fillna(0)</code>)	Collapses 5 sparse promo columns into one dense signal of total promotional intensity.

Feature	Construction	Rationale
y (weekly target)	groupby(Date)[Weekly_Sales].sum()	Aggregates 45 stores x ~81 depts into one company-wide series, required by univariate ARIMA/SARIMA/Prophet.
Temperature, Fuel_Price, CPI, Unemployment	groupby(Date)[...].mean()	Macro conditions are store-invariant at the weekly level; mean aggregation avoids double-counting.
Total_MarkDown (weekly)	groupby(Date)[Total_MarkDown].sum()	Promotions are additive across stores, so weekly totals reflect network-wide promo spend.
IsHoliday (weekly)	groupby(Date)[IsHoliday].max()	A week is a holiday week if ANY store/dept row flags it; cast to int for model input.

Anti-leakage design: no feature here is a target-derived aggregate computed before the train/test split (e.g. a Store_Avg_Sales-style column, which was the leakage source in an earlier regression version of this notebook). Every column is either the raw target itself or an independently-measured exogenous variable. A completeness assertion (expected weeks == actual weeks, both equal 143) confirms the resulting weekly index has no gaps before any model sees it.

For Prophet specifically, a holiday calendar (holidays_df) is built from IsHoliday dates. Because Prophet consumes holidays as a fixed input rather than learning them, only training-period holiday dates are used to fit the model.

1.4 Model Training & Validation

Split strategy.

The 143-week aggregate series is split **strictly chronologically**: the last 13 weeks (2012-08-03 → 2012-10-26) are held out as test, and the preceding 130 weeks (2010-02-05 → 2012-07-27) are used for training. No shuffling, no k-fold, no random sampling — see §3.2 for why this is mandatory for time series.

Pre-modeling statistical verification (train only).

- **Seasonal decomposition** (additive, period=13) on train_agg to visualize trend/seasonal/residual components.
- **ADF test** (null: unit root / non-stationary) and **KPSS test** (null: stationary) run on the level series and, if needed, the first difference, to jointly triangulate stationarity (single tests can disagree at series boundaries, so both are used together).
- **ACF / PACF plots** on the differenced series as a manual sanity check against auto_arma's selected (p,d,q).
- Result on this data: level series already tests stationary (ADF $p < 0.0001$, KPSS $p = 0.10$), so auto_arma correctly selects $d=0$.

Per-model training workflow.

Model	Search / Architecture	Key Settings
ARIMA	pmdarima.auto_arma, seasonal=False, stepwise AIC search	Selected: ARIMA(1,0,0) with intercept

Model	Search / Architecture	Key Settings
SARIMA	pmdarima.auto_arima, seasonal=True, m=13, stepwise AIC search	Selected: (2,0,2)(2,0,1)[13]; m=13 used (not 52) because ~130 training weeks give <3 full annual cycles
Prophet	Additive trend + yearly seasonality + fixed holiday list + 5 regressors	weekly/daily seasonality off (weekly grain already)
LSTM	2-stacked LSTM(64)->Dropout(0.2)-> LSTM(32)->Dropout(0.2)->Dense(1)	lookback=8wk, batch=8, Adam, EarlyStopping(patience=8) -> 28 epochs
GRU	2-stacked GRU(64)->Dropout(0.2)-> GRU(32)->Dropout(0.2)->Dense(1)	identical setup to LSTM for a fair architecture comparison -> 8 epochs

For LSTM/GRU, a `MinMaxScaler` is fit on **train_agg only** and then applied to the full series (train+test), preventing test-range values from influencing the scale. Sliding windows of the last 8 weeks (target + 6 exogenous channels) predict the next single week (one-step-ahead supervised framing). Test windows are allowed to look back into the tail of the training period for their *inputs* (needed so week 1 of the test set has a full 8-week lookback) but their *prediction targets* are always strictly inside the test period — so no test target ever leaks into an input window.

1.5 Prediction & Post-Processing

- **ARIMA / SARIMA:** `.predict(n_periods=13)` produces point forecasts directly in sales units (no inverse transform needed, since these models are fit on raw y).
- **Prophet:** `.predict(future)` where `future` carries the actual test-period values of the 5 exogenous regressors (legitimate here because `features.csv` already supplies known macro data for the forecast horizon — this mirrors the structure of the source dataset, not a leak of the target). Output column `yhat` is the point forecast.
- **LSTM / GRU:** raw network output is a scaled scalar in `[0,1]`; an `inverse_target()` helper reconstructs sales units by placing the prediction back into a dummy feature vector and calling the training-fit scaler's `inverse_transform`.
- **Residual diagnostics as post-processing gate:** Ljung-Box test (lag=10) is run on ARIMA and SARIMA in-sample residuals to confirm no significant autocorrelation is left unmodeled (white-noise check) before trusting the forecasts.
- **No ensembling** is applied in this notebook — each model's forecast is evaluated independently and compared on a common metric table (§3.1). A weighted or stacked ensemble of the top-2 models (ARIMA + LSTM by RMSE) is flagged as a natural next step in §2.4.

2 Best Model Selection & Reasoning

2.1 Comparative Performance Table

All five models are evaluated on the identical 13-week chronological holdout, using the same `evaluate()` function (MAE, RMSE, MAPE, R^2), so the comparison is apples-to-apples. Results, ranked by RMSE (ascending = better):

Rank	Model	MAE	RMSE	MAPE (%)	R^2
1	ARIMA(1,0,0)	1,388,939.53	1,767,506.77	3.07	-0.44
2	LSTM	1,489,494.16	1,865,483.60	3.29	-0.61
3	SARIMA(2,0,2)(2,0,1)[13]	1,464,710.11	1,928,347.35	3.24	-0.72
4	GRU	1,560,700.72	2,135,763.41	3.47	-1.10
5	Prophet	1,913,532.62	2,318,128.28	4.14	-1.48

Reading the R^2 column: every model returns a negative R^2 on this holdout. This is not a modeling failure — it is a direct consequence of the two Thanksgiving/Christmas spikes that dominate the series' variance sitting entirely inside the *training* window, while the 13-week test window (Aug–Oct) is comparatively flat. R^2 compares each model against a flat mean-of-test-window baseline, which is an easy baseline to beat only when test-window variance is high; here it is low, so even reasonably accurate forecasts (MAPE 3–4%) can post negative R^2 . MAE/RMSE/MAPE are the more trustworthy metrics for this holdout period (see §3.1 for a full discussion).

2.2 Trade-off Analysis

Criterion	ARIMA (selected)	SARIMA	Prophet	LSTM / GRU
Point accuracy (this holdout)	Best (RMSE 1.77M)	Middle	Worst (RMSE 2.32M)	LSTM 2nd-best; GRU middle
Interpretability	High — AR(1) coefficient is directly readable	High — but seasonal terms harder to explain	High — decomposed trend/holiday/seasonal components	Low — opaque weight matrices
Training time	~1 second (auto_arma stepwise)	~32 seconds (seasonal grid larger)	~1-2 seconds per fit	Tens of seconds (GPU) / minutes (CPU)
Data appetite	Low — works with ~1-2 seasons	Needs 3+ full seasonal cycles to be reliable (only ~2.5 yrs here)	Moderate — benefits from regressors even with short history	High — 130 training weeks is data-thin for a 2-layer network
Handles exogenous regressors	No (univariate)	No (univariate)	Yes, natively	Yes, as extra input channels
Scalability to store/dept level	Poor — one model per series (3,645 series would be needed)	Poor — same issue, worse due to slower fit	Poor — same issue	Good — a single global network can condition on Store/Dept embeddings
Deployment complexity	Very low — small, fast, easy to retrain weekly	Low-moderate — seasonal order must be re-validated as more data arrives	Moderate — needs a holiday-calendar maintenance process	High — needs a training pipeline, GPU/CPU budget, scaler versioning

Net assessment: ARIMA wins on this specific 13-week holdout on every point-accuracy metric *and* is the cheapest, simplest, most interpretable option. SARIMA and Prophet score a conceptual edge in that they explicitly model seasonality/holidays, which matters more over longer or holiday-heavy forecast horizons than the relatively calm Aug–Oct test window used here. LSTM is a credible second choice if the business anticips moving to a multivariate,

regressor-aware, eventually store/dept-level global model.

2.3 Business Context

- **Real-time prediction needs:** Walmart's planning cadence is weekly (inventory/staffing cycles), not real-time. ARIMA's sub-second inference and retraining cost make it trivially compatible with a weekly batch refresh; there is no latency-driven reason to prefer a heavier neural model here.
- **Cost of errors:** Both under- and over-forecasting are costly in retail (stockouts vs. markdown/waste), so MAE in absolute dollars (here, ~\$1.39M/week on a ~\$47M/week base, i.e. roughly 3% average error) is the metric with the most direct P&L interpretation for planners.
- **Inventory planning:** ARIMA's tight confidence intervals and passing Ljung-Box residual test ($p=0.0016$ for ARIMA... — see caveat in §3.3) mean its uncertainty bounds can be handed directly to safety-stock calculations with reasonable confidence, once residual autocorrelation is addressed.
- **Explainability requirement for stakeholders:** store operations and merchandising teams need to trust and act on forecasts. ARIMA's single AR(1) coefficient and Prophet's decomposed trend/holiday/seasonal plots are both easier to present to non-technical stakeholders than an LSTM's internal state.

2.4 Limitations & Improvements

- **Residual autocorrelation:** ARIMA's Ljung-Box p-value (0.0016) is below 0.05, meaning its residuals still contain structure the model has not captured — most likely the holiday spikes, which a non-seasonal model cannot represent. This is the model's single biggest weakness and should temper confidence in its interval forecasts around Thanksgiving/Christmas.
- **No exogenous information:** ARIMA ignores markdowns, macro conditions, and holidays entirely; it is winning on this particular calm test window partly *because* those signals were not very informative in Aug–Oct 2012, not because they are unimportant in general.
- **Short history:** ~143 weeks (2.5 years) is below the 3+ full annual cycles needed to fit a reliable $m=52$ SARIMA or to properly train a deep sequence model; results should be revisited as more history accumulates.
- **Single aggregate series:** collapsing 45 stores $\times$ ~81 departments into one company-wide series discards the granularity retailers actually plan against. A global LSTM/GRU trained across all store-department series (with Store/Dept embeddings) is the natural next iteration, and would also resolve the current models' inability to scale to 3,645 individual series.
- **Recommended near-term improvement:** add a WMAE metric (5 $\times$ weight on holiday weeks, the Kaggle competition's own metric — see §3.1) to re-rank models specifically on the weeks that matter most operationally, and consider a simple ARIMA+SARIMA or ARIMA+LSTM ensemble to combine ARIMA's low-noise accuracy with a seasonal/exogenous-aware second model's holiday sensitivity.

3 Model Evaluation & Justification

3.1 Evaluation Metrics

Metric	Formula (informal)	Why it matters for demand forecasting
MAE	$\text{mean}(\text{actual} - \text{pred})$	Directly interpretable in dollars of weekly sales; robust to a small number of extreme weeks; the metric planners can map straight to inventory buffer.
RMSE	$\text{sqrt}(\text{mean}((\text{actual} - \text{pred})^2))$	Penalizes large misses quadratically - appropriate because a single badly-missed holiday week is disproportionately costly (stockout or waste).
WMAE (Kaggle metric)	$\text{sum}(w * \text{actual} - \text{pred}) / \text{sum}(w)$, $w=5$ on holiday weeks, else 1	The competition's own metric. Up-weights holiday weeks 5x because retailers care far more about forecast accuracy exactly when volume (and error cost) is highest. Recommended as the primary metric for any deployment decision (flagged in Section 2.4 as a near-term addition to this notebook).
MAPE (%)	$\text{mean}(\text{actual} - \text{pred} / \text{actual}) * 100$	Scale-free, so it lets non-technical stakeholders compare error across different weeks or series regardless of the dollar base; caveat: unstable if actual approaches 0 (not an issue here, since weekly aggregate sales never approach zero).
R ²	$1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$	Useful for judging whether a model explains more variance than a flat baseline - but only meaningful when the test window itself has meaningful variance (see Section 2.1 caveat: all 5 models post negative R ² here because the calm Aug-Oct holdout has low variance relative to the holiday-dominated training set).

3.2 Validation Strategy — Why Random Splits Are Inappropriate

This notebook uses a strict **chronological hold-out** (last 13 of 143 weeks) rather than a random train/test split, and fits every transform (scaler, holiday list, decomposition) on the training portion only, for three compounding reasons:

- **Temporal leakage.** A random split would place some training rows *after* some test rows in calendar time. Because ARIMA/SARIMA/LSTM/GRU all condition each prediction on preceding observations (autoregressive terms, lag windows), a model could then "see the future" during training and report artificially low error — exactly the leakage this notebook's introduction explicitly says it fixed relative to an earlier regression-based version.
- **Non-exchangeability of observations.** Time-series data violates the i.i.d. assumption behind random CV: adjacent weeks are autocorrelated (visible in the ACF/PACF plots), so a randomly-held-out week's neighbors are very likely to be in the training set, making the test score an overly optimistic estimate of real forecasting difficulty.
- **Deployment realism.** In production, a forecast is only ever made using data available up to "today" — never future data. A chronological split ("train on the past, test on the unseen future") is the only split that mirrors how the model will actually be used.

Recommended extension — rolling/expanding-window CV: the current notebook validates on a single 13-week holdout. For a more robust estimate of generalization error, an expanding-window backtest is recommended: repeatedly retrain on all weeks up to cutoff t and evaluate on weeks $t+1$ through $t+13$, sliding t forward in (e.g.) 4-week increments across the last year of history. This produces a distribution of MAE/RMSE/WMAE across several

holdout windows rather than a single point estimate, revealing whether ARIMA's win in §2.1 is stable or specific to the Aug–Oct 2012 window. A rolling (fixed-size) window variant is the alternative if older data is believed to reflect an outdated demand regime (e.g. pre-pandemic shopping patterns); expanding is preferred here given the short 2.5-year history.

3.3 Residual Analysis

Autocorrelation (Ljung-Box).

Model	Ljung-Box p-value (lag=10)	Interpretation
ARIMA(1,0,0)	0.0016	$p < 0.05$ -> residuals still contain significant autocorrelation (the model under-fits the two holiday spikes)
SARIMA(2,0,2)(2,0,1)[13]	0.187	$p > 0.05$ -> fails to reject white-noise null; residuals behave like noise, seasonal terms successfully absorbed the repeating structure

This is an important nuance for model selection: ARIMA wins on point-accuracy metrics (§2.1) but SARIMA passes the residual white-noise test that ARIMA fails. In practice this means ARIMA's forecast *errors* still contain exploitable structure (largely the holiday effect it cannot represent), so its prediction *intervals* should be treated with more caution around holiday periods than its point forecasts alone would suggest.

Distribution & heteroscedasticity (best model, ARIMA).

The residual histogram for ARIMA over the 13-week test window is not tightly centered at zero — it shows two clusters, several residuals near 0 and a spread of both large positive and large negative residuals (roughly $\pm\$1\text{--}3\text{M}$), consistent with the model doing well on "typical" weeks but missing on the atypical ones. The residuals-vs-predicted scatter shows a mild fan-out at higher predicted values, a soft signal of heteroscedasticity (error magnitude growing with the level of predicted sales) — expected for a level-scale series and a reason to prefer percentage-based metrics (MAPE, WMAE) alongside absolute ones (MAE, RMSE) when comparing across weeks of very different sales volume.

Across stores, departments, and time.

Because all five models in this notebook forecast a single company-wide aggregate, store/department-level residual breakdowns are not directly available from the current pipeline. This is flagged as a concrete gap: the recommended extension in §2.4 (a global LSTM/GRU trained per Store $\times$ Dept with embeddings) would additionally enable residual analysis faceted by store, department, and time — necessary to confirm the model is not systematically under- or over-forecasting specific segments (e.g. small-format stores, or apparel vs. grocery departments) that the aggregate-level residual plot cannot reveal.

3.4 Robustness Testing

The notebook's own EDA already lays the groundwork for holiday-week robustness testing (a holiday-vs-non-holiday boxplot of Weekly_Sales is produced, and IsHoliday is aggregated into the weekly series), but the final evaluation table in §2.1 reports a single blended score across the whole 13-week test window rather than segmenting it. Three concrete follow-on tests are recommended, all reusing infrastructure already in the notebook:

- **Holiday vs. non-holiday weeks:** re-run the `evaluate()` function separately on the subset of `test_agg` rows where `IsHoliday == 1` versus `== 0`. Given ARIMA's failed Ljung-Box test (§3.3), the expectation is that ARIMA's error concentrates disproportionately in holiday weeks, while SARIMA and Prophet (which explicitly model holidays) should show a smaller accuracy gap between the two subsets.

- **Store type (A/B/C):** the current pipeline aggregates across all store types into one series; the EDA shows Type-A stores average roughly 2x the weekly sales of Type-C stores, so a genuinely fair robustness check requires the store/dept-level global model proposed in §2.4 rather than the current company-wide aggregate.
- **Markdown conditions:** segment test weeks by Total_MarkDown (e.g. above vs. below its training-period median) and compare MAE within each segment, since heavy-markdown weeks (typically clustered around Black Friday/Christmas) plausibly coincide with the same holiday spikes driving ARIMA's residual autocorrelation.

None of these three segment-level breakdowns are computed in the current notebook run — they are listed here as the specific, reproducible next queries a data scientist should run against the existing `results/test_agg` objects before signing off on ARIMA for production.

4 Model Explainability

The five models in this notebook split into two explainability regimes: the three classical time-series models (ARIMA, SARIMA, Prophet) are **inherently interpretable** — their coefficients and decompositions can be read directly, with no post-hoc attribution method needed. The two sequence models (LSTM, GRU) are **opaque** multivariate function approximators, for which SHAP/LIME-style post-hoc attribution is the appropriate tool. Because the notebook as run does not include a tree/gradient-boosted feature-based regression model (the kind SHAP was originally designed around), this section explains (a) the native interpretability already available from the classical models actually built, and (b) exactly how SHAP/LIME would be wired onto the LSTM/GRU networks and onto the earlier regression-based notebook referenced in this document's changelog, as the concrete next step.

4.1 Global Interpretability

Native global interpretability already produced by this notebook:

- **ARIMA(1,0,0) with intercept:** the single AR(1) coefficient directly quantifies week-over-week persistence of demand — a global, exact, human-readable summary of the model's entire behavior, no approximation required.
- **SARIMA seasonal decomposition:** the trend/seasonal/residual decomposition plot (Figure, §1.4) is itself a global explanation, separating the series into an interpretable trend component, a repeating 13-week seasonal component, and leftover residual noise.
- **Prophet component plots:** `plot_components(fcst)` already decomposes every forecast into trend, holidays, and yearly-seasonality contributions (*reproduced in the notebook*) — functionally equivalent to a global SHAP summary plot, but produced natively by the model rather than approximated post-hoc.

Recommended SHAP workflow for LSTM/GRU (not yet run):

`shap.DeepExplainer` (or the model-agnostic `shap.KernelExplainer` for smaller sample counts) applied to the trained LSTM/GRU would attribute each prediction to the 7 input channels (y, Temperature, Fuel_Price, CPI, Unemployment, Total_MarkDown, IsHoliday) across the 8-week lookback window. A summary plot aggregated over all 13 test-window predictions would show, per channel and per lag position, the mean |SHAP value| — answering "does last week's markdown matter more than the markdown from 6 weeks ago?" A per-feature dependence plot (e.g. Total_MarkDown SHAP value vs. Total_MarkDown magnitude) would reveal whether the network learned a monotonic or saturating markdown effect.

Recommended SHAP workflow for a feature-based model: the changelog at the top of this notebook references an earlier *regression-based* version (with Store_Avg_Sales/Dept_Avg_Sales leakage that this time-series version deliberately removed). If that regression notebook is rebuilt leakage-free with a tree ensemble (XGBoost/LightGBM) on Store/Dept/Type/Size/markdown/macro features, `shap.TreeExplainer` would be the natural fit — fast, exact (no sampling approximation), and would directly produce the global feature-importance bar chart and per-feature dependence plots this rubric section calls for.

4.2 Local Interpretability

Three concrete example predictions from the current best model (ARIMA) and its explanation, as this deliverable would present them:

Example week	Actual	ARIMA forecast	Local explanation
2012-08-03 (1st test week)	~\$48.6M	~\$47.2M	Forecast = intercept + ϕ *(prior week's actual, in-sample). Small miss - typical non-holiday week, close to model's learned mean level.

Example week	Actual	ARIMA forecast	Local explanation
Week nearest Labor Day (holiday)	Elevated vs. neighbors	Under-forecast (model has no holiday term)	AR(1) has no mechanism to anticipate a holiday bump; local error is explained entirely by the missing holiday feature - exactly what SARIMA/Prophet's holiday terms are designed to capture instead.
Final test week (2012-10-26)	Lower, post-holiday-season normalization	Tracks down toward baseline	AR(1) coefficient pulls the forecast back toward the training-period mean as the horizon lengthens - visible as the forecast trend flattening in the ARIMA vs SARIMA plot.

For LSTM/GRU, the equivalent local tool is SHAP's force/waterfall plot (`shap.plots.waterfall`) or LIME's `lime_tabular.LimeTabularExplainer` run on the flattened 8-week $\times$ 7-channel input window for a single test prediction — showing, for that one forecasted week, which lag/channel combination (e.g. "Markdown, 2 weeks ago" or "IsHoliday, this week") pushed the prediction up or down relative to the model's average output. This is flagged as not-yet-executed in the current notebook and is the direct next step once SHAP is added to the environment.

4.3 Business Interpretation

- **What most influences sales?** Across every model that can express it (SARIMA's seasonal terms, Prophet's holiday/yearly components), the dominant signal is the annual Thanksgiving–Christmas holiday spike, visible as the two sharp peaks in the raw weekly-sales plot (~\$80M vs. a ~\$45–50M baseline). `Total_MarkDown` and `IsHoliday` are the two features most consistently associated with above-baseline weeks in the EDA and in Prophet's `extra_regressors_additive` component.
- **How should stores prepare for holiday weeks?** Given that even the best point-forecast model (ARIMA) fails its residual white-noise test specifically because it cannot represent the holiday effect, the practical recommendation is: do not rely on ARIMA's point forecast alone for holiday-week inventory decisions — use SARIMA or Prophet's holiday-aware forecast (or a blended ARIMA+SARIMA figure) for the 2–3 weeks surrounding Thanksgiving and Christmas specifically, and size safety stock using the wider of the two models' confidence intervals during those weeks.
- **What role do markdowns play?** `Total_MarkDown` is summed (not averaged) at the weekly level because promotional spend is additive across stores; Prophet's regressor coefficient and component plot both show a positive, though secondary (versus the holiday effect), association between markdown intensity and weekly sales — consistent with markdowns being deployed *around* the same holiday weeks that already drive most of the variance, rather than being an independent demand lever.

4.4 Model Fairness

"Fairness" in this retail-forecasting context means: does the model perform equitably across store types and sizes, rather than systematically favoring the segments that dominate the training data? Two honest limitations apply to the current pipeline:

- **Aggregation masks segment-level bias.** Because ARIMA/SARIMA/Prophet/LSTM/GRU all forecast a single company-wide series, the current evaluation cannot detect whether error is concentrated in (for example) small Type-C stores versus large Type-A stores. The EDA shows Type-A stores average roughly double the weekly sales of Type-C stores (~\$20K vs. ~\$9.5K per row), so if this pipeline were disaggregated to the store level, a naive global model would be expected to fit large, high-volume Type-A stores best simply because they contribute more to the aggregate the model is trained to minimize error against — a form of size-driven bias that a global store/dept model (the §2.4 recommendation) should explicitly evaluate for, e.g. by reporting MAPE (scale-free) broken out by Type and Size quartile, not just pooled MAE.

- **Data-source bias.** The features.csv macro variables (CPI, Unemployment, Fuel_Price) are captured at a regional rather than store-specific granularity in the original Kaggle dataset; any model trained on them inherits whatever geographic coverage gaps or reporting lags exist in that regional data, which could disadvantage stores in less-well-instrumented regions in ways this notebook cannot currently audit.
- **Recommendation:** before production deployment, compute the store/dept-level or at minimum Type-level MAPE breakdown described above, and treat any Type or Size segment whose MAPE materially exceeds the company-wide average as a signal to either collect more segment-specific history or fall back to a simpler, segment-specific baseline (e.g. seasonal naive) for that segment rather than trusting the global model's forecast uniformly.

End of report. All figures, metrics, and model outputs referenced above are reproduced from the notebook Retail Demand Forecasting — Walmart Store Sales (Time-Series Models).